

Adversarial Test-Hardening for AI-Written Code: A Pre-Registered Replication, a Null Result, and an Instrument Autopsy

Jeff Otterson (Independent)

July 2026

Disclosure: This manuscript was drafted by an AI assistant from the author’s frozen, pre-registered experimental artifacts under the author’s direction; every quantitative and design claim was checked against the committed sources. The tooling is disclosed rather than hidden, consistent with the paper’s subject (see Section 3.9).

Abstract

Large language models increasingly write both code and the tests meant to check it, and coverage, the usual measure of a suite, records what ran rather than what was verified. We study an adversarial test-hardening loop under a mechanical oracle: a Tester model writes tests, mutation testing names the surviving injected defects, and a Critic model writes tests to kill exactly those, with every verdict decided mechanically so that no model judges another model’s output. In a pre-registered experiment on five Python subjects (one same-lineage-loop cell could not be scored) we test whether the loop kills more mutants than one-shot generation (H1) and whether a cross-lineage Critic outperforms a same-lineage one (H2), reporting cost per killed mutant. H1 is supported and is a replication of a known effect in a new setting: the loop killed 105 mutants that one-shot generation missed and lost none. H2 is a pre-declared, underpowered null. The central contribution is an autopsy: an earlier analysis reported a cross-lineage effect at $p = 9.5 \times 10^{-66}$ that was a lineage-correlated instrument artifact, an output cap that silently truncated the verbose model, caught only by adversarial review of the completed analysis and collapsing to non-significance once corrected. We conclude that cross-model comparisons can inherit the asymmetries of the harness that runs them, and we release the protocol, receipts, and analysis.

1. Introduction

Large language models now write both production code and the tests meant to check it, and the second job is the weaker link. Coverage, the usual measure of a test suite, records which lines ran, not whether anything about their behavior was actually verified, and a model asked to write tests for its own code tends to take the path of least resistance: it exercises the happy path and asserts weakly, producing suites that are green and nearly empty of fault-detecting power. A test suite can be fully covered and catch almost nothing. As models take over more of the testing work, the evidentiary value of “the tests pass” is arguably falling, precisely where practitioners are relying on it more.

Mutation testing is the standard way to measure a suite’s fault-detecting power directly. It injects small, deliberate defects (mutants) into the code and counts how many the suite catches: a killed mutant is a defect the tests detected, and a surviving mutant is a specific, named gap the tests would let through. We use mutation testing as a mechanical oracle inside a two-role loop. A Tester model writes an initial suite; the surviving mutants are handed, by name, to a Critic model that writes tests to kill exactly those; the loop repeats until no new mutants die. Every verdict in this loop is mechanical: a generated test either kills a mutant under the test runner or it does not, and no model ever judges its own or another model’s output. The design deliberately removes model judgment from the verdict, which is the channel through which self-preference bias corrupts model-graded evaluation. Removing model judgment from the verdict is necessary but, as this study’s own experience shows (Section 5), not sufficient to make a cross-model comparison neutral.

We do not claim this loop is new. Feeding survivor mutants back into a model prompt and running an adversarial test-versus-mutant loop under a mechanical oracle are both established in prior work. Our study is a pre-registered empirical evaluation, and its value is in the discipline with which it was run and reported, not in the idea. We froze the hypotheses, the subjects, the measurement scopes, and the success criteria before any paid run, enforced that freeze in tooling, and required every reported number to be recomputable from committed per-run receipts. We test two hypotheses on five pinned Python subjects under the mutation-kill oracle: H1, that the adversarial loop kills more mutants than one-shot generation; and H2, that a cross-lineage Critic (a different-provider model) kills more of the survivors a same-lineage Tester missed than a same-lineage Critic does. We report cost per killed mutant throughout.

H1 is supported and, we argue, is a clean replication of a known effect in a new agentic, repo-level, Python setting: across the paired mutants where the two approaches disagreed, the loop killed 105 that one-shot generation missed and lost none, a result that grew stronger, not weaker, when a measurement defect working against it was corrected. H2 is not supported; it is a pre-declared and honestly underpowered null, because most subjects saturated at the kill ceiling and left almost nothing to discriminate on.

The paper’s central contribution is neither of those results. It is an account of how this study’s own measurement instrument manufactured a large false result and how that was caught. An earlier version of the H2 analysis reported an overwhelming cross-lineage effect, a pooled significance of $p = 9.5 \times 10^{-66}$. It was almost entirely an artifact. A fixed output ceiling on one provider’s model silently truncated that model’s replies, and because the same-lineage Critic is the verbose model and the cross-lineage Critic is terse, the truncation was systematically correlated with the exact variable under test. It did not add noise; it manufactured a difference. The defect produced no crash and no error, only a plausible rejection, and it survived every mechanical gate in the pipeline; it was caught only by a final adversarial review of the completed analysis, and when it was corrected the apparent effect collapsed to non-significance. The general lesson, which this paper documents with receipts, is that a cross-model comparison can inherit the asymmetries of the harness that runs it: closing the model-judgment channel does not make the comparison neutral if the apparatus feeding the two models is not itself symmetric.

This paper makes four contributions:

1. A pre-registered replication of the adversarial test-hardening effect (H1) in a new agentic, repo-level, Python setting, with per-outcome costs disclosed, an accounting we believe is the first of its kind in the LLM test-generation literature.
2. A pre-declared, honestly underpowered null on the cross-lineage Critic question (H2) under a

mechanical mutation-kill oracle, a comparison we could not find measured in prior work.

3. As the durable contribution, a fully receipted autopsy of a lineage-correlated instrument artifact that manufactured an extraordinary false result, together with the fail-closed instrumentation that detects and prevents its recurrence where a mechanical output cap is declared, and the general lesson for cross-model evaluation that follows from it.
 4. A public, reproducible artifact: the pre-registered protocol, the per-run receipts, the deviations log, and the analysis script that recomputes every reported statistic.
-

2. Related Work

2.1 LLM-based test generation

A large recent literature uses large language models to generate unit tests directly. TestPilot [arXiv:2302.06527, TSE 2024] generates JavaScript tests from signatures and documentation usage examples with no additional training; ChatTester [arXiv:2305.04207] and ChatJUnit [arXiv:2305.04764] add generate-then-repair loops that fix compilation and assertion errors in model-written tests. For Python specifically, CoverUp [arXiv:2403.16218, FSE 2025] interleaves coverage analysis with iterative model dialogs to target uncovered lines, and CodaMosa [ICSE 2023], building on the search-based generator Pynguin (Lukasczyk and Fraser), queries a model to re-seed search when coverage plateaus. HITS [arXiv:2408.11324, ASE 2024] decomposes complex methods into slices for higher coverage. At industrial scale, TestGen-LLM [arXiv:2402.09171, FSE 2024] extends existing test classes only when a generated test verifiably builds, passes reliably, and increases coverage, using that filter to eliminate hallucinated tests. These systems establish that models can write useful tests and that a filter is needed to trust them, but they gate on coverage or on compile-and-pass repair, not on fault detection. Our work differs in the gate: it scores generated tests by the mutants they kill, a stronger signal than coverage, and it is the adversarial survivor-feedback loop, not one-shot generation, that our H1 evaluates. This study does not include a non-LLM search-based baseline such as Pynguin or EvoSuite; the comparison of interest here is one-shot versus adversarial-loop LLM generation, not LLM versus search-based generation.

2.2 Mutation testing with language models

Mutation testing supplies the mechanical oracle for this work, and several systems combine it with language models. MuTAP [arXiv:2308.16557; IST 2024] re-prompts a single model with its surviving mutants (produced by the rule-based tool MutPy) to generate tests that kill them, halting when no survivors remain; it is single-agent, benchmark-scale, and reports mutation score with no dollar cost. Meta ACH [arXiv:2501.12862, FSE 2025] uses three agents (fault generation, equivalence detection, and test generation) with a single backbone model to produce few, concern-specific mutants and hardening tests at industrial scale, under an assured generate-and-test filter. LLMorpheus [arXiv:2404.09952] instead generates the mutants themselves with a model, producing diverse bug-like mutants beyond a standard operator set, and a recent empirical study [arXiv:2406.09843, TOSEM] compares such model-generated mutants against real bugs at scale. Our mutants are rule-based (mutmut), as in MuTAP, rather than model-generated as in LLMorpheus and AdverTest; this keeps the injected defect independent of any model under test.

2.3 Adversarial and dual-agent test-versus-mutant loops

The closest neighbor is AdverTest [arXiv:2602.08146], which runs a genuine dual-agent adversarial loop: a test agent and a mutant agent iteratively attack each other under an execution-based mutation-kill oracle, with measured fault-detection gains over prior baselines. Crucially, both of AdverTest’s agents share a single backbone model per configuration, and no cross-lineage configuration is measured. Our loop replicates this adversarial-under-a-mechanical-oracle pattern, and we claim it as a replication, not an invention (Section 5.1). What AdverTest and the survivor-feedback lineage of MuTAP do not do, and what our H2 measures, is vary the lineage of the critic against a same-lineage control under the mechanical oracle.

2.4 Cross-model diversity, self-preference, and evaluation bias

The premise that a mechanical oracle is worth its cost rests on a known failure of model-graded evaluation: large language model evaluators can recognize and favor their own generations [Panickssery, Bowman, and Feng, arXiv:2404.13076, NeurIPS 2024], a self-preference bias a model-graded verdict cannot escape. Heterogeneous-model methods such as multi-agent debate [Du et al., arXiv:2305.14325, ICML 2024] motivate the intuition that mixing model lineages helps, but the direction is genuinely open: Self-MoA reports same-model repeated sampling beating heterogeneous mixtures on reasoning benchmarks, and Refute-or-Promote [arXiv:2604.19049] argues a cross-model critic catches correlated blind spots but does so in defect discovery, without a controlled same-lineage ablation and without a purely mechanical oracle. Most directly relevant to our Discussion, recent work argues that models increasingly share the same errors, which undermines the reliability of using one model to oversee another [Great Models Think Alike and this Undermines AI Oversight, arXiv:2502.04313]. Our instrument autopsy (Section 5) is a concrete, receipted instance of a related but distinct hazard: even with model judgment removed from the verdict entirely, an asymmetry in the harness that feeds the two models can manufacture a large false cross-model difference.

2.5 Cost accounting and pre-registration

Where cost is reported in this literature, it is normalized to an input unit rather than an outcome. AdverTest reports an average dollar cost per method-generation run (for example \$0.270 on Defects4J), and TestForge [arXiv:2503.14713] reports dollars per file and per iteration, but neither divides cost by mutants killed or faults detected. The nearest per-outcome figure is in an adjacent domain, defect discovery, where a cross-model critic system reports a per-discovered-vulnerability cost. To our knowledge our cost-per-kill is the first per-outcome cost figure reported in LLM test generation. Separately, pre-registered and registered-report protocols have precedent in software engineering [arXiv:2302.03649] and have begun to wrap LLM-for-software-engineering studies [arXiv:2606.10702], though they remain uncommon in this literature; we adopt pre-registration as a matter of rigor and claim no priority for it.

2.6 Positioning

To our knowledge, no prior work measures the effect of critic-generator lineage diversity on test-suite fault detection under a mechanical mutation-kill oracle, nor reports LLM test-generation cost normalized per killed mutant. We do not claim to be first to run an adversarial test-versus-mutant loop under a mechanical oracle (AdverTest does this), first to feed survivor mutants into model prompts (MuTAP does this), or first to pre-register a software-engineering protocol. We claim a

controlled cross-lineage versus same-lineage critic measurement under a mechanical oracle, a per-outcome cost figure new to this literature, a replication of the MuTAP-to-AdverTest direction in an agentic repo-level Python setting with disclosed costs, and, as the durable contribution, a fully receipted autopsy of a lineage-correlated instrument artifact.

3. Method

3.1 The adversarial test-hardening loop

We evaluate a two-role loop over a mechanical mutation-kill oracle. A Tester model is prompted once to write a pytest suite for a target module (round 0). The suite is run under mutation testing (mutmut): each surviving mutant is an injected defect the suite failed to catch. A Critic model is then handed the named survivors, the specific mutants left alive together with their diffs, and prompted to write tests that kill exactly those. The loop repeats. After each Critic round the suite is re-measured, newly killed survivors are removed from the target list, and the next round attacks what remains. A run stops when two consecutive rounds kill nothing new (`dry_rounds = 2`) or after four rounds (`max_rounds = 4`), whichever comes first.

The verdict at the core of this loop is mechanical: a generated test either kills a mutant under pytest or it does not, and no model ever judges its own or another model’s output. This closes the self-preference channel that undermines LLM-as-judge evaluations, where a model scoring model output has no ground truth. We are careful not to overstate what this buys. The mechanical oracle removes model judgment from the verdict itself; it does not make the surrounding measurement apparatus neutral. Section 5 documents a case where the harness around the oracle opened a lineage-correlated channel into the result even though the oracle itself never rendered a subjective judgment.

We do not claim the loop is novel. Feeding survivor mutants into an LLM prompt was established by MuTAP, and a dual-agent adversarial test-versus-mutant loop under a mechanical oracle was established by AdverTest. Our contribution is a controlled measurement, described below, and a fully receipted account of what it took to make that measurement trustworthy (Section 5).

3.2 Arms and hypotheses

Each subject module is run through three arms, all sharing one fixed Tester model (claude-sonnet-5, Anthropic) so that the only deliberate variable is the Critic:

- oneshot: Tester round 0 only, no Critic round. This is the one-shot baseline.
- loop-same: Tester round 0, then Critic rounds where the Critic is the same lineage as the Tester (claude-sonnet-5, Anthropic).
- loop-cross: Tester round 0, then Critic rounds where the Critic is a different lineage (gpt-5.6-terra, OpenAI).

The cross-lineage Critic is deliberately price- and capability-tier matched to the same-lineage Critic (\$2.50/\$15 per million tokens versus \$3/\$15) rather than paired with the flagship variant, so that the comparison isolates lineage rather than confounding it with model capability tier. We pin real model identifiers throughout, rather than anonymizing, because the cost accounting is tied to specific rate cards and the instrument finding in Section 5 is provider-specific and cannot be stated without them.

Two hypotheses follow:

- H1 (replication): the adversarial loop (loop-same) kills more mutants than one-shot generation (oneshot), holding the Tester model fixed.
- H2 (novel claim): a cross-lineage Critic (loop-cross) kills more of the survivors the same-lineage Tester missed than a same-lineage Critic (loop-same) does.

H1 is framed strictly as a replication of the MuTAP-to-AdverTest loop direction in a new agentic, repo-level, Python setting with disclosed per-outcome costs. H2 is the claim we could not find measured in prior work: to our knowledge no prior study measures the effect of critic-generator lineage diversity on test-suite fault detection under a mechanical mutation-kill oracle. Both directions of the H2 result were pre-declared publishable, including a null, so the direction was not assumed in our favor.

One design limitation is inherent to these arms and we state it where the arms are defined rather than burying it later. Each arm independently regenerates its own round 0, so the tester suite and its resulting survivor set are not held fixed between loop-same and loop-cross. The two critic arms therefore differ in both the round-0 tester draw and the critic lineage, and single-run tester draws carry substantial variance (Section 6 reports the same tester producing round-0 kill counts of 69, 168, 50, 62, and 65 across repeated attempts on one subject). A cleaner design would freeze one round-0 survivor set and run both critics from that identical state; this study did not, and any small cross-lineage difference it reports must be read against that confound.

3.3 Subjects and selection

Five Python subjects, each pinned by commit SHA, each contributing one paired subject-module cell to every arm. Subjects were selected against criteria fixed before selection (recorded in the pre-registration’s selection log). The two third-party subjects were drawn by scanning the most-downloaded PyPI packages in rank order and taking the first two that met every pre-registered criterion with no discretion: pure Python (no C extensions), a permissive license, an existing pytest suite (which we strip in the clone), a plain-logic module of 100 to 800 source lines (no network- or IO-heavy modules; the first qualifying module in alphabetical order becomes the target), at least 40 mutants generated by mutmut on that module, and no prior authorship, contribution, or analysis by the author. The three author-repository subjects are a convenience sample chosen for thesis relevance, including the degenerate zero-kill baseline case described below. This convenience component is a disclosed threat to external validity (Section 6), compounded by training-data contamination since all five are public code the models may have seen in pretraining. For each subject exactly one target module is mutated, and sibling files needed for imports are carried into the mutation sandbox unmutated. One subject (graph-guard) was pre-declared as a pilot with a go/no-go gate before the full grid, and its pilot cells were pre-declared to count as data; in the event those cells were later reclassified for an instrument defect (Section 3.8) and rerun, so the pilot designation does not affect the counted dataset.

Subject	Provenance	Target module	Suite	Mutants	Killed by existing suite	Baseline survivors (scored)
attrition-risk-ml	author repo (public)	src/train.py	py	255	0	255

Subject	Provenance	Target module	Suite	Mutants	Killed by existing suite	Baseline survivors (scored)
graph-guard	author repo (public)	<code>graph_guard/loop.py</code>	<code>graph_guard/loop.py</code>	80	58	22
rag-guard	author repo (public)	<code>rag_guard/loop.py</code>	<code>rag_guard/loop.py</code>	71	46	25
packaging	pypa/packaging (OSS)	<code>packaging/_elffile.py</code>	<code>packaging/_elffile.py</code>	69	0	69
idna	kjd/idna (OSS)	<code>idna/cli.py</code>	<code>idna/cli.py</code> stripped	187	0	187

Both mutation denominators are shown, per the pre-registration’s rule that neither the full-mutant count nor the baseline-survivor count is presented alone. The two third-party subjects have their existing suites stripped in the local clone before any run, so the loop is scored against a genuinely empty starting suite and never against the upstream project’s own tests; the three author-repo subjects keep their existing suites, so their scored denominator is the survivors an already-present suite left alive.

One subject, attrition-risk-ml, is degenerate by design and kept deliberately. Its existing suite kills 0 of 255 mutants, because the target module is never imported by any test, so it is the maximal-headroom false-pass case: a suite that looks present but catches nothing, exactly the failure mode this line of work exists to expose. Because any ratio against a zero-kill baseline is undefined and any intervention trivially wins against zero, this subject is reported with absolute kill counts only, never a relative-improvement figure.

We perform no equivalent-mutant detection. Some mutants in each subject may be semantically equivalent to the original and therefore unkillable by any test, which means every reported kill rate is a lower bound. This does not affect the paired comparisons: both arms in any comparison face the identical mutant set, so an unkillable mutant simply lands in the both-miss cell that McNemar discards.

3.4 The mechanical oracle

Mutation testing is run with mutmut, which injects rule-based, single-change mutants (one mutation per variant), in contrast to LLM-generated mutants used by some adversarial-loop systems. Each subject’s mutation scope, meaning the exact module mutated and the exact sibling files copied into the sandbox, was fixed at pre-registration and thereafter changed only through logged, receipted amendments (Section 3.8), never silently at run time. Kills are counted per mutant over each subject’s own pristine-baseline survivor set: round 0’s survivors measured before any generated test exists, so both arms in any comparison are scored against the same universe of mutants. A generated test that fails to compile, fails on the pristine (un-mutated) module, or fails a flake check (it must pass on pristine code twice) is rejected before it can contribute a kill; the rejection and its reason are recorded, and the round credits zero kills. A test that passes collection but fails on the pristine module, a wrong-oracle test rather than a bad file, is pruned individually and logged, never counted as a kill.

3.5 The paired statistic

Because every arm runs against the same pinned subject clone, kill outcomes are paired mutant-for-mutant across arms. The primary statistic is a two-sided exact McNemar test on the discordant pairs: b = mutants killed by arm A only, c = mutants killed by arm B only. Pairs where both arms agree (both kill, or both miss) carry no information and are excluded by construction; the evidence lives entirely in the disagreements. The test uses the doubled exact binomial tail (min-tail doubling, with $n = 0$ defined as $p = 1$), not the chi-square approximation, which is appropriate for the small discordant counts these subjects produce.

The pooled test treats individual mutants as the unit of analysis, which assumes independence across mutants; in fact mutants are clustered by subject (idna alone contributes 64 of the 105 H1 discordant pairs), and within-subject correlation among mutants of one module is not modeled. This inflates the nominal pooled significance, which is why we pre-registered per-subject tables as a companion readout and report them alongside every pooled figure rather than resting on the pool alone.

Each hypothesis is reported in three pre-declared views, all fixed before any data. First, one pooled McNemar across all subjects' discordant pairs, designated the confirmatory statistic. Second, a pooled view excluding the degenerate subject, whose 255 paired outcomes would otherwise dominate the pool. Third, per-subject McNemar tables. All three views were committed in advance and all three are reported for both hypotheses, so no view can be chosen, promoted, or dropped after seeing results. We ran no a priori power analysis; with three of four scorable subjects at or near a 98.6 to 100 percent kill ceiling, the number of discordant pairs available to H2 was near its structural minimum, so the H2 null is stated throughout as "no effect detectable at this design's power," never as "no effect."

3.6 Pre-registration, enforced by tooling

The protocol, meaning the hypotheses, arms, subjects, frozen scopes, the exact statistic, and the supported-or-not-supported criteria, was written and committed before any paid run. The freeze is not a promise. The experiment command mechanically refuses to run any arm unless the committed machine-readable protocol is byte-identical to the repository HEAD, and every reported number is required to be recomputable from committed per-run receipts (the per-round JSON logs recording token counts, cost, verdicts, and preserved test artifacts for every run). The success criteria were fixed in advance: H1 is supported if and only if the pooled McNemar yields $p < 0.05$ and the direction favors loop-same; H2 if and only if the pooled McNemar yields $p < 0.05$ and the direction favors loop-cross. Any other outcome, in either case, is not supported, and a null was pre-declared to be reported with the same prominence as a positive result. No subgroup analysis was permitted to substitute for the three pre-declared views; any post-hoc breakdown is labeled exploratory and never replaces the primary readout.

Each arm runs once per subject under a fixed grid (three arms times five subjects, fifteen cells). A cell that produces a valid verdict is never rerun in search of a better one, and there is no early stopping for significance. A cell that is invalidated or aborts may be rerun only under a logged deviation, and one subject's loop-same cell was attempted five times against a pre-committed attempt budget; when that budget was exhausted without a valid verdict, the cell was recorded as missing data rather than backfilled or estimated. Every departure from the frozen protocol is logged in an append-only deviations file. Fourteen of the fifteen cells produced valid verdicts; the one missing cell (attrition-risk-ml loop-same) removes the largest subject from the paired statistics, so

all paired analyses span four subjects, and the pre-declared with-degenerate and without-degenerate pooled views coincide in composition for this dataset (a degeneracy we disclose rather than smooth over).

3.7 Cost accounting

Every model round is metered exactly from its input and output token counts; an unpriced model fails the run closed rather than pricing at a guessed rate. We report cost-per-kill, meaning total cost divided by mutants killed, per arm. To our knowledge this per-outcome normalization is not reported in the LLM test-generation literature, where systems report cost per method-run or per file; the adjacent defect-discovery literature does report a per-outcome figure (a cross-model critic system reports a dollar cost per discovered vulnerability), so we scope the novelty to test generation rather than claiming it broadly.

3.8 Instrument integrity

The measurement instrument was itself treated as a subject of scrutiny, and the honest account of what that surfaced falls into three tiers rather than one.

First, defects caught for free, before any model was called, by a model-free dry-run validator built partway through the study: scope-transcription errors and a mutation-sandbox failure that would have silently recorded every mutant as unchecked. From the point that validator existed, and later a canary must-kill probe that confirms the sandbox actually collects a freshly written test, each subsequent scope fix was validated by a zero-cost dry run before any further paid cell ran.

Second, and this is the part a paper about instrument honesty must not soft-pedal, defects caught only after paid cells had already run, which forced reclassification of already-spent data. A trampoline crash that laundered into a false all-survived zero, and an include-list scoping bug that silently prevented freshly generated tests from being collected, together invalidated several paid cells, including two that had previously been counted as data. More was ultimately spent on runs that were invalidated and rerun than on the runs finally counted. That the instrument had to throw away paid data is not the embarrassing part; concealing it would be.

Third, the one defect caught only after all fifteen counted cells had run, by a final adversarial review of the completed analysis rather than by any runtime gate. Because it changed the headline result, it is documented in full as a finding in its own right (Section 5). In every tier the repair principle was the same: make the instrument fail loudly rather than record a plausible-looking wrong number. Across the study the protocol carried ten numbered amendments plus one prompt-template amendment, each logged with its rationale and its cost consequences.

3.9 Reproducibility and disclosure

All counted cells ran under a single fixed prompt version, with one per-subject import hint added for the one subject whose package layout required it; the prompt texts are content-hashed and the hashes are recorded in the pre-registration (16-character prefixes: tester 98abbb8532990865, critic 703335c0af111836). The models were called at provider default sampling: no temperature or top-p value is set anywhere in the experiment’s provider configuration, and the only sampling-relevant request-body parameter is the output-token cap discussed in Section 5. Mutation testing used mutmut 3.6.0 (pinned) with pytest 8 or newer; the package requires Python 3.11 or newer and its continuous integration validates on Python 3.11, 3.12, and 3.13. All runs executed on macOS

on Apple Silicon, the platform whose fork behavior produced the missing-cell deadlock analyzed in Section 6. The protocol, the machine-readable arms configuration, the per-run receipts, the deviations log, and the analysis script that recomputes the reported statistics are committed to the public repository, so every reported statistic is reproducible from the receipts. A small, bounded portion of the all-in spend total is disclosed as an estimate rather than receipt-derived, where a crash billed a call before its receipt row landed. An automated reproduction command is provided. The manuscript was drafted by an AI assistant from these frozen artifacts under the author’s direction, and every quantitative and design claim was checked against the committed sources; this tooling is disclosed rather than hidden, consistent with the paper’s subject. The pre-registration’s limitations (training-data contamination, no syscall sandbox against mutant-environment gaming, a two-run flake check that a one-third-flaky test still clears roughly four times in nine, a single fixed Tester model, one subject’s stdin code path, and one subject’s pinned-module divergence from its development branch) are carried into a dedicated Threats to Validity section rather than asserted away here.

4. Results

Every figure below is recomputed from the committed per-run receipts by the analysis script, driven by a committed manifest of which runs count. This analysis supersedes an earlier one: a final adversarial review of the completed analysis found a truncation defect sitting inside the counted cells (an output cap with no truncation detection), which reclassified four counted cells as instrument-invalid and confounded the entire H2 readout. The version reported here is computed from the corrected instrument with the affected cells rerun. The headline difference is itself the paper’s central finding: H2’s previously significant result did not survive the correction, and the Discussion analyzes why.

4.1 Counted cells

The design is five subjects times three arms, fifteen cells. Fourteen produced valid verdicts; one, attrition-risk-ml loop-same, is a missing cell, after its original run was reclassified instrument-invalid and four rerun attempts each failed on a nested-parallelism deadlock that survived three targeted, individually verified instrument fixes, exhausting the pre-committed attempt budget. Kill counts are per mutant over each subject’s own pristine-baseline survivor set. The verdict column records how each run terminated: oneshot (baseline, no critic round); clean (all baseline survivors killed); dry (two consecutive rounds killed nothing new); rounds-cap (reached the four-round limit); missing (no valid verdict within the attempt budget). Dropped counts wrong-oracle tests pruned across all rounds and never credited as kills.

Subject	Arm	Verdict	Baseline survivors	Killed	Rate	Cost (\$)	Dropped
graph-guard	oneshot	oneshot	22	12	54.5%	0.13	0
graph-guard	loop-same	dry	22	17	77.3%	1.02	1
graph-guard	loop-cross	clean	22	22	100.0%	0.35	3

Subject	Arm	Verdict	Baseline survivors	Killed	Rate	Cost (\$)	Dropped
rag-guard	oneshot	oneshot	25	21	84.0%	0.06	0
rag-guard	loop-same	clean	25	25	100.0%	0.11	0
rag-guard	loop-cross	clean	25	25	100.0%	0.16	1
packaging	oneshot	oneshot	69	36	52.2%	0.11	0
packaging	loop-same	rounds-cap	69	68	98.6%	0.64	0
packaging	loop-cross	dry	69	68	98.6%	0.30	3
idna	oneshot	oneshot	187	123	65.8%	0.12	0
idna	loop-same	clean	187	187	100.0%	0.65	9
idna	loop-cross	clean	187	187	100.0%	0.19	0
attrition-risk-ml	oneshot	oneshot	255	62	24.3%	0.07	0
attrition-risk-ml	loop-same	missing	255	n/a	n/a	0.07	0
attrition-risk-ml	loop-cross	rounds-cap	255	254	99.6%	0.34	2

Costs are shown to two decimals for readability; the aggregate figures in Section 4.5 are computed from the full-precision receipts, so summing the rounded column will not exactly reproduce them. Nineteen wrong-oracle tests were pruned across the fourteen valid cells (nine of them in idna loop-same alone), each logged and never counted as a kill.

Because attrition-risk-ml’s loop-same cell is missing, that subject contributes no paired data to either hypothesis, so all paired statistics below span the remaining four subjects. The pre-declared pooled-with-degenerate and pooled-without-degenerate views therefore coincide in composition for this dataset, a degeneracy we disclose rather than smooth over.

4.2 H1 (replication): does the loop kill more than one-shot?

Pre-written support criterion (fixed before any run): H1 is supported if and only if the pooled McNemar test comparing loop-same to oneshot yields $p < 0.05$ and the direction favors loop-same ($b > c$, where b counts mutants killed by loop-same only).

Per-subject paired 2x2, over each subject’s baseline survivor set:

Subject	both	b (loop-same only)	c (oneshot only)	neither	p (exact McNemar)
graph-guard	12	5	0	5	0.0625

Subject	both	b (loop-same only)	c (oneshot only)	neither	p (exact McNemar)
rag-guard	21	4	0	0	0.125
packaging	36	32	0	1	4.66e-10
idna	123	64	0	0	1.08e-19

Pooled (confirmatory view): $b = 105$, $c = 0$, $p = 4.93\text{e-}32$.

Verdict: H1 SUPPORTED. The pooled p is far below 0.05 and the direction favors loop-same. The sharper statement is the discordant split itself: across 303 paired baseline survivors in four subjects, not one mutant was killed by one-shot generation but missed by the loop. Two subjects individually clear $p < 0.05$ (packaging, idna) and two do not (graph-guard at 0.0625, rag-guard at 0.125, both with small discordant counts), but the direction is consistent in all four. The pooled test treats mutants as independent when they in fact cluster by subject, which inflates the nominal pooled significance; the pool is also idna-heavy (64 of the 105 discordant kills) and packaging-heavy (32 of 105). We therefore report the per-subject tables alongside the pool rather than resting on the pooled figure.

Notably, the instrument correction strengthened H1. Under the defect-affected instrument the H1 split read $b = 79$, $c = 5$, with one subject tied and one point against the loop. The truncation defect had been silently removing critic rounds that loop-same should have run, suppressing the loop’s own kills. Correcting it moved packaging from a 4-to-4 tie to plus 32 and graph-guard from 0-versus-1-against to plus 5 (one accepted critic round of over 30,000 output tokens, impossible under the old cap, killed 5 survivors). The defect worked against H1’s own hypothesis, so the true effect could only be as large or larger than the defect-affected data showed.

4.3 H2 (novel claim): does a cross-lineage critic kill more than a same-lineage critic?

Pre-written support criterion: H2 is supported if and only if the pooled McNemar comparing loop-cross to loop-same yields $p < 0.05$ and the direction favors loop-cross. A null was pre-declared to be reported with the same prominence as a positive result.

Per-subject paired 2x2:

Subject	both	b (loop-cross only)	c (loop-same only)	neither	p (exact McNemar)
graph-guard	17	5	0	0	0.0625
rag-guard	25	0	0	0	1.0 (no discordant pairs, ceiling)
packaging	68	0	0	1	1.0 (no discordant pairs)
idna	187	0	0	0	1.0 (no discordant pairs, ceiling)

Pooled (confirmatory view): $b = 5$, $c = 0$, $p = 0.0625$.

Verdict: H2 NOT SUPPORTED. The pooled p of 0.0625 does not clear the pre-registered 0.05 bar. The point direction favors loop-cross (all five discordant pairs, from the single subject graph-guard, go the cross-lineage way, and none go the other way), but the evidence does not meet the pre-written criterion. This is an underpowered null, not a demonstration of no effect: three of the four scorable subjects saturated at 98.6 to 100 percent kill in both loop arms, leaving zero discordant pairs, so the design had almost no room to detect a cross-lineage difference even if one exists. The correct reading is no effect detectable at this design’s power, with a directionally positive but non-significant residual on one subject.

4.4 The load-bearing finding: H2’s earlier signal was an instrument artifact

Reported at the same prominence as any supported result, because it is the paper’s central contribution. The superseded analysis reported H2 SUPPORTED at a pooled p of $9.5e-66$, with $b = 217$ and $c = 0$, an apparently overwhelming cross-lineage effect. The corrected instrument erased essentially all of it.

- packaging: b went from 32 to 0. The old loop-same cell’s two critic rounds had both been silently truncated at exactly 16,000 output tokens each and rejected under unrelated-sounding notes, crediting zero kills. Rerun with truncation detection and a raised cap, the same-lineage critic’s rounds were accepted and killed 32 more survivors, landing at 68 of 69, identical to loop-cross. The strongly supported reading is that the cap deleted the verbose model’s rounds; because this rests on a single rerun of a cell class that carries real run-to-run variance, tester-round variance as a partial contributor cannot be fully excluded by the receipts, so this is the well-supported reading rather than a mechanical decomposition.
- attrition-risk-ml: b went from 185 to excluded. Its old loop-same cell was likewise truncation invalidated, and the subject could not produce a valid replacement within the attempt budget, so the missing-cell rule removed it from the comparison entirely.
- What remains is graph-guard’s $b = 5$, a real gap between two accepted runs worth noting descriptively, and three subjects at or near ceiling with no discordant pairs at all.

Stated plainly: the dramatic cross-lineage effect this experiment appeared to measure was manufactured by a lineage-correlated instrument failure and does not survive the instrument’s correction. The output cap sat on every call to the same-provider model, so it truncated the verbose same-lineage critic and some tester rounds alike, never the terse cross-lineage critic; the bias it injected into H2 specifically ran through the same-lineage critic rounds, which is what manufactured the apparent cross-lineage gap. The Discussion analyzes the mechanism, the fail-loud repair, and why this generalizes to any cross-model comparison run through an asymmetric harness.

4.5 Cost per kill

Aggregated across counted cells with valid verdicts (total cost summed and divided by total kills summed per arm, from full-precision receipts, never averaged as a per-subject ratio):

Arm	Total cost (\$)	Total killed	Aggregate \$/kill
oneshot	0.4844	254	0.0019
loop-same	2.4170	297	0.0081
loop-cross	1.3293	556	0.0024

One asymmetry must be read alongside the 3.4-times gap between loop-cross and loop-same: the oneshot and loop-cross aggregates each span all five subjects, while loop-same spans only its four valid cells (the missing attrition-risk-ml cell). loop-cross’s 556 kills therefore include attrition-risk-ml’s 254, which loop-same never reached, so the arms are aggregated over different subject sets and the ratio is not a like-for-like per-subject comparison.

With that caveat, the loop-cross arm is about 3.4 times cheaper per kill than loop-same, and the instrument correction changed the reason. In the superseded analysis the gap was largely same-lineage dollars spent on truncation-rejected rounds that credited zero kills. With truncation fixed, the residual gap reflects output-length economics: the same-lineage critic writes long replies (up to roughly 30,000 output tokens per accepted round, billed at \$15 per million output tokens) while the cross-lineage critic writes terse replies (a few hundred to a few thousand output tokens per accepted round) at a similar rate card. The honest reading is that verbosity, not lineage capability, is the dominant cost variable between these two critics. To our knowledge this per-outcome cost figure is the first reported in the LLM test-generation literature.

5. Discussion

5.1 What H1 establishes, and what it does not

H1 is a replication, and we are careful to claim it as one. The result that the adversarial loop strictly dominates one-shot generation, killing 105 discordant mutants to zero across four subjects, confirms in a new setting (agentic, repo-level, Python, with a mechanical mutation-kill oracle) an effect already established by MuTAP and AdverTest. Replication in a new setting is a contribution, but it is not a discovery, and the strength of the pooled p-value does not change that. The value of this particular replication is narrower and more concrete: it comes with disclosed per-outcome costs, and it survived an instrument-correction process that, as it happened, made the effect larger rather than smaller (Section 4.2), because the defect described below had been suppressing the loop’s own rounds. An effect that grows when you fix the instrument working against it is a more trustworthy effect than one that shrinks.

5.2 The instrument autopsy

The paper’s central contribution is not H1 and not the loop. It is a fully receipted account of how this study’s own measurement instrument manufactured a large, false result, how that result was caught, and what it cost to catch it.

What appeared. Before the correction, H2 read as strongly supported: a pooled exact McNemar of $p = 9.5e-66$, with 217 discordant pairs favoring the cross-lineage critic and none against. On its face this was an overwhelming demonstration that a different-lineage critic outperforms a same-lineage one. It was almost entirely false.

The defect. The provider wrapper for the same-lineage model set a fixed output ceiling (`max_tokens = 16000`) on every one of its calls, and nowhere in the calling code was a reply’s output-token count ever compared against that ceiling, nor was the truncation signal the API returns ever inspected. A reply that hit the ceiling came back as ordinary, well-formed-looking text that happened to be cut off mid-file. At the call site it was indistinguishable from a complete reply. It was handed to test validation, which rejected it for an unrelated-sounding reason (a truncated code block reads as “expected exactly one fenced python block, found 0”; a syntactically

broken test file reads as “invalid: fails on pristine code”), and the round was recorded as rejected under that misleading note, crediting zero kills, with no trace anywhere in the receipts that the reply had in fact been silently cut off, billed, and discarded. Across the counted cells, eight rounds were recorded rejected, and seven of them had billed exactly 16,000 output tokens.

Why this faked a cross-lineage effect rather than adding noise. This is the crux, and it is what makes the defect a confound rather than random error. The truncation was not evenly distributed across the two critic arms. The same-lineage critic is the verbose model: its replies, which walk through each survivor’s diff and reasoning, routinely approached or exceeded the 16,000 token ceiling. The cross-lineage critic is terse and never hit any ceiling in any counted cell (its accepted-round output ranged from a few hundred to a few thousand tokens). So the truncation failure was systematically correlated with exactly the variable H2 was measuring. It did not blur the comparison; it manufactured one. The same-lineage arm was the only one mechanically capable of losing whole rounds to silent truncation, which biased the measurement toward making the same-lineage critic look worse than its real output, for a reason having nothing to do with test quality or lineage. When the defect was corrected and the four affected cells rerun, the apparent effect collapsed: the pooled p of $9.5e-66$ became $p = 0.0625$, not significant. The clearest single illustration is packaging, whose 32-to-0 discordant gap favoring the cross-lineage critic became 0-to-0 after its same-lineage critic’s rounds were rerun without truncation. Because that rests on a single rerun of a cell class with real run-to-run variance, the well-supported reading is that the ceiling had been deleting the verbose model’s rounds, rather than a mechanical decomposition that fully excludes ordinary tester-round variance.

How it was caught. Not by a runtime gate. This study caught three earlier silent-corruption mechanisms mechanically, each before any number reached a results file: a sandbox crash that would have laundered into a false all-survived zero, and a scoping bug that would have silently prevented freshly generated tests from ever being collected, among them. The truncation defect slipped all of those gates, because it produced no crash and no error, only a plausible rejection note. It was caught by a final adversarial review that read the completed analysis, after every counted cell had already run, and asked why exactly these rounds were failing. That a defect this consequential survived every mechanical check and was caught only by an independent skeptical read is itself part of the finding.

The fail-loud repair. The fix makes the instrument halt loudly rather than emit a plausible wrong number. The output ceiling is now a single mechanical value read by both the request and a new check: every reply’s output-token count is compared against it, and a reply at the ceiling raises a truncation error that is never retried (retrying a capped call would bill again and likely truncate again). The round is recorded honestly as “truncated: output hit max_tokens cap,” its real billed cost is metered, zero kills are credited, and the raw truncated reply is archived as preserved evidence. A truncation at the first round of a cell fails that cell loudly as missing data rather than being silently recovered. The principle is fail-closed: when the instrument cannot measure, it must say so, not hand back a confident wrong number.

A residual asymmetry we disclose rather than paper over. The truncation check exists only where a mechanical ceiling is declared, which is the same-provider wrapper. The cross-lineage provider sends no such ceiling and is therefore never truncation-checked. If that provider ever truncated a reply against a server-side limit, this instrumentation would launder it exactly the way the original defect did. We have no evidence any such truncation occurred (every accepted cross-lineage round is far from any plausible ceiling), but the detection asymmetry is structural and we name it, because the entire point of this section is that undisclosed asymmetries are how

false results get made.

5.3 The general lesson: cross-model comparisons can inherit the asymmetries of their harness

The mechanical oracle at the core of this design was adopted specifically to close the self-preference channel that undermines LLM-as-judge evaluation, where a model scoring model output has no ground truth. At the verdict level it did exactly that: no model ever judged another model’s output, and the kill verdict is a seeded defect and a failing test. The autopsy’s payload is that a different bias, one level out from the verdict, survived the very mechanism meant to make model-versus-model comparison safe. Closing the judgment channel did not make the comparison neutral, because the apparatus that fed the two models was not itself symmetric.

The lesson generalizes beyond the specific defect. A comparison is only as neutral as the apparatus that runs it, and an apparatus that treats the two models differently in any dimension correlated with the outcome can produce a difference that looks like a model difference and is not. Here the correlated dimension was output verbosity interacting with a truncation ceiling; in another study it could be a timeout, a token budget, a rate limit, a retry policy, or a parser that tolerates one model’s formatting quirks and not the other’s. The defect is dangerous precisely because it is invisible in the result: the manufactured p-value was clean, well-formatted, and enormous. Nothing about the number announced that it was measuring the harness rather than the models.

Two practices follow, and this study argues for both by having needed both. First, pre-registration of the hypotheses and success criteria, so that a result cannot be quietly reinterpreted after the fact and so that the difference between the pre-declared analysis and the corrected one is itself a visible, dated record. Second, and less commonly practiced, fail-closed instrumentation: a measurement pipeline should be built so that when a component cannot do its job, it halts loudly instead of emitting a plausible substitute. Most of the silent corruptions in this study were caught because the instrument was progressively rebuilt to fail loud; the one that reached the counted data was the one place the instrument could still fail silently.

5.4 Why the null and the autopsy are the contribution

It would have been easy, and wrong, to publish the $p = 9.5e-66$ result. It was pre-registered, it was in the right direction, and it was produced by real runs against real subjects. A study that stopped at its first significant result would have reported a large cross-lineage effect that the corrected data do not support. What prevented that was not a better model or a cleverer method; it was an evidence discipline that treated the instrument as a suspect and subjected the completed analysis to an adversarial read before publication. The replication (H1) confirms known work. The null (H2), honestly reported as underpowered rather than as evidence of no effect, is the pre-declared second finding. But the durable contribution is the autopsy: a worked, receipted example of a lineage-correlated instrument artifact manufacturing an extraordinary false result, caught before publication, with the fail-loud repair that prevents its recurrence. Cross-model evaluation is now routine in software-engineering research and practice; this study documents, with receipts, one concrete way such evaluations silently go wrong, and what it takes to catch it.

A properly powered test of the cross-lineage question would need three things this study lacked: subjects with survivor headroom below the 98.6 to 100 percent kill ceiling that left three of four scorable subjects with no discordant pairs, repeated cells per subject to separate signal from the substantial run-to-run tester variance, and a single frozen round-0 survivor set shared by both critic

arms, which would remove the tester-draw confound noted where the arms are defined (Section 3.2). The limitations of the study, including its small subject count, its single language, its one missing cell, the ceiling effects, and the residual detection asymmetry named above, are treated in full in the Threats to Validity section.

6. Threats to Validity

6.1 Construct validity

The oracle measures mutant kills, which is a proxy for fault detection, not fault detection itself. Two gaps follow. First, we perform no equivalent-mutant detection, so some counted survivors may be semantically equivalent to the original and unkillable by any test; every reported kill rate is therefore a lower bound. This does not distort the paired comparisons, because both arms in any comparison face the identical mutant set and an unkillable mutant falls in the both-miss cell that McNemar discards, but it does mean absolute kill rates should be read as floors. Second, we do not run generated tests inside a syscall sandbox, so a sufficiently adversarial generated test could in principle detect properties of the mutation harness itself (timing, file paths, environment markers) and pass or fail on that rather than on the mutant’s behavior. Our guardrails (add-only tests, pass-on-pristine validation, a flake check, and wrong-oracle pruning) reduce this but do not mechanically close it. A third construct threat concerns the cost measure: cost-per-kill does not isolate lineage, because the same-lineage critic’s cost is dominated by its output verbosity rather than its lineage, so the cross-arm cost gap reflects model output style as much as the critic mechanism.

6.2 Internal validity

The H2 comparison carries a confound we disclose where the arms are defined (Section 3.2): each arm independently regenerates its own round-0 tester suite, so loop-same and loop-cross differ in both the tester draw and the critic lineage, and a single-run tester draw carries substantial variance (the same tester produced round-0 kill counts of 69, 168, 50, 62, and 65 across repeated attempts on one subject). A small cross-lineage difference could therefore reflect tester noise rather than lineage; a cleaner design would share one frozen round-0 survivor set across both critic arms.

The study’s central internal-validity event is the instrument artifact analyzed in Section 5: an output cap that silently truncated the verbose same-lineage critic and manufactured a large false H2 effect before it was caught and corrected. We report it as a finding rather than hiding it, but one residual remains: truncation is mechanically detected only for the provider that declares an output cap, so a server-side truncation on the other provider would still be laundered. We have no evidence any occurred (every accepted cross-lineage round is far from any plausible cap), but the detection asymmetry is structural. A further, smaller asymmetry: one subject’s missing-cell rerun attempts ran with an added parallelism-control configuration that its other two arms did not, because that configuration was introduced to fight a deadlock; the kill oracle is a deterministic pass or fail with no channel from a parallelism backend to a verdict, but the instrument configuration across that subject’s arms is not byte-identical, and we record it. Finally, all five subjects are public code that the models may have seen in pretraining; the mutant-kill metric blunts but does not eliminate contamination, since a memorized suite would still have to target the injected mutant to score a kill.

6.3 External validity

The strongest limits are on generalization. One cell, the same-lineage-loop arm of the largest subject (255 baseline survivors), failed to produce a valid verdict within the pre-committed attempt budget, and the fixed once-per-subject design has no backfill mechanism, so that subject is dropped from both hypotheses entirely and the pre-declared with-degenerate and without-degenerate pooled views collapse to a single four-subject composition; the miss is disclosed, not smoothed over. The remaining paired statistics therefore span four subjects, of which three are the author’s own public repositories, a convenience sample; the study is single-language (Python) and holds a single tester model fixed across all arms, so it does not speak to whether a different tester lineage would change either result. Ceiling effects further bound what H2 could show: three of the four scorable subjects saturated at 98.6 to 100 percent kill in both loop arms, leaving no headroom to detect a cross-lineage difference. Two subject-specific notes also bear on generalization: one subject’s target module reads standard input in its command-line path, which mutation and collection never exercise, and one subject’s pinned module differs from its own development-branch version, so any future run against that branch would be a different subject, not a rerun.

6.4 Statistical conclusion validity

The pooled McNemar test treats individual mutants as independent units, but mutants cluster by subject (one subject supplies 64 of the 105 H1 discordant pairs), and within-subject correlation is not modeled; this inflates the nominal pooled significance, which is why per-subject tables are pre-registered and reported alongside every pooled figure. No a priori power analysis was performed, and the H2 result is an underpowered null: with so few discordant pairs available, $p = 0.0625$ is near the smallest two-sided value the design could produce, so the null means no effect detectable at this power, never no effect. Kill counts also carry real run-to-run variance from single-run cells, so per-subject point estimates should be read as noisy; for example, one subject’s same-lineage-loop kills moved from 11 of 22 to 17 of 22 between its invalidated original run and its valid rerun. And the flake check accepts a test after it passes on pristine code twice; a test that is flaky roughly one third of the time still clears a two-run check roughly four times in nine, so some residual flaky-kill noise is expected and is not separately modeled.

References

The inline markers of the form [arXiv:XXXX] in the text identify the entries below.

- TestPilot: M. Schafer, S. Nadi, A. Eghbali, F. Tip. “An Empirical Evaluation of Using Large Language Models for Automated Unit Test Generation.” *IEEE Transactions on Software Engineering*, vol. 50 no. 1, pp. 85-105, 2024. arXiv:2302.06527; DOI 10.1109/TSE.2023.3334955.
- ChatTester: “No More Manual Tests? Evaluating and Improving ChatGPT for Unit Test Generation.” arXiv:2305.04207.
- ChatUniTest: Y. Chen et al. “ChatUniTest: A Framework for LLM-Based Test Generation.” arXiv:2305.04764.
- CoverUp: J. A. Pizzorno, E. D. Berger. “CoverUp: Coverage-Guided LLM-Based Test Generation.” arXiv:2403.16218; *Proc. ACM Softw. Eng. (FSE 2025)*, DOI 10.1145/3729398.
- CodaMosa: C. Lemieux, J. P. Inala, S. K. Lahiri, S. Sen. “CodaMosa: Escaping Coverage Plateaus in Test Generation with Pre-trained Large Language Models.” *ICSE 2023*.

- Pynguin: S. Lukasczyk, G. Fraser. “Pynguin: Automated Unit Test Generation for Python.” ICSE-Companion 2022, pp. 168-172. arXiv:2202.05218; DOI 10.1145/3510454.3516829.
- HITS: Z. Wang, K. Liu, G. Li, Z. Jin. “HITS: High-coverage LLM-based Unit Test Generation via Method Slicing.” ASE 2024. arXiv:2408.11324.
- TestGen-LLM: N. Alshahwan et al. “Automated Unit Test Improvement using Large Language Models at Meta.” FSE 2024 (Industry). arXiv:2402.09171; DOI 10.1145/3663529.3663839.
- MuTAP: A. M. Dakhel, A. Nikanjam, F. Khomh, M. C. Desmarais, H. Washizaki. arXiv:2308.16557; Information and Software Technology, 2024, DOI 10.1016/j.infsof.2024.107468.
- Meta ACH: C. Foster, A. Gulati, M. Harman, I. Harper, K. Mao, J. Ritchey, H. Robert, S. Sengupta. “Mutation-Guided LLM-based Test Generation at Meta.” FSE 2025 (Industry). arXiv:2501.12862.
- LLMorpheus: F. Tip, J. Bell, M. Schafer. “LLMorpheus: Mutation Testing using Large Language Models.” arXiv:2404.09952.
- Comprehensive Study: B. Wang, M. Chen, M. Deng, Y. Lin, M. Harman, M. Papadakis, J. M. Zhang. “A Comprehensive Study on Large Language Models for Mutation Testing.” arXiv:2406.09843; TOSEM.
- AdverTest: Chang, Fang, Chen, Shi, Shen, Gu. “Test vs Mutant: Adversarial LLM Agents for Robust Unit Test Generation.” arXiv:2602.08146.
- A. Panickssery, S. R. Bowman, S. Feng. “LLM Evaluators Recognize and Favor Their Own Generations.” NeurIPS 2024. arXiv:2404.13076.
- Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, I. Mordatch. “Improving Factuality and Reasoning in Language Models through Multiagent Debate.” ICML 2024. arXiv:2305.14325.
- Self-MoA: W. Li, Y. Lin, M. Xia, C. Jin. “Rethinking Mixture-of-Agents: Is Mixing Different Large Language Models Beneficial?” 2025. arXiv:2502.00674.
- Refute-or-Promote: A. Agarwal. “Refute-or-Promote: An Adversarial Stage-Gated Multi-Agent Review Methodology for High-Precision LLM-Assisted Defect Discovery.” 2026. arXiv:2604.19049.
- Great Models Think Alike: S. Goel, J. Struber, I. A. Auzina, K. K. Chandra, P. Kumaraguru, D. Kiela, A. Prabhu, M. Bethge, J. Geiping. “Great Models Think Alike and this Undermines AI Oversight.” 2025. arXiv:2502.04313.
- TestForge: K. Jain, C. Le Goues. “TestForge: Feedback-Driven, Agentic Test Suite Generation.” 2025. arXiv:2503.14713.
- EvoSuite: G. Fraser, A. Arcuri. “EvoSuite: Automatic Test Suite Generation for Object-Oriented Software.” ESEC/FSE 2011, pp. 416-419. DOI 10.1145/2025113.2025179.
- Registered Reports in Software Engineering: N. A. Ernst, M. T. Baldassarre. Empirical Software Engineering, 2023. arXiv:2302.03649.
- Watts and Debts: A. S. Shany, S. Chandrasekar, K. Vaidhyanathan. “Watts and Debts of Agentic Frameworks: An Empirical Study (Registered Report).” ESEM 2026 (Registered Reports Track). arXiv:2606.10702.